CO5-AT2: STUDENT RESULT MANAGEMENT SYSTEM (JDBC)

PROGRAM TERMINAL OUTPUT

Student Result JDBC Output

```
--- Query 1: Fetch by Student ID (101) ---
[JDBC Query] SELECT * FROM students WHERE student_id = 101
UGStudent [ID=101  | Arun Kumar      | Dept=CSE          | Scholarship=false]
Calculated Grade: A+

--- Query 2: Fetch by Department (CSE) ---
[JDBC Query] SELECT * FROM students WHERE department = 'CSE';
UGStudent [ID=101  | Arun Kumar      | Dept=CSE          | Scholarship=false]
Calculated Grade: A+

--- Query 3: Catch Custom Exception Demo (ID=105) ---
[JDBC Query] SELECT * FROM students WHERE student_id = 105
Successfully caught exception: Negative or invalid marks error: Eve Davis (Marks: -5.0)

c:\java studies\Java>
```

SQL DATABASE TABLE (DB BROWSER FOR SQLITE)

DB Browser for SQLite - C:\java studies\Java\students.db

File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes | Undo | Open Project | Save Project | Attach Database | Close Database

Database Structure **Browse Data** Edit Pragmas Execute SQL

Table: students Filter in any column

	student_id	name	department	type	marks	has_scholarship
1	101	Arun Kumar	CSE	UG	88.5	0
2	102	Bhavna R	ECE	UG	92.0	1
3	103	Chaitanya S	AI-DS	PG	84.0	1
4	104	Deepak N	MECH	UG	76.5	0
5	105	Eve Davis	IT	PG	-5.0	0

JAVA SOURCE CODE

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class DatabaseOperationException extends Exception {
    public DatabaseOperationException(String msg) {
        super(msg);
    }
}

abstract class Student {
    protected int studentId;
    protected String name;
    protected String department;
    protected double marks;
    protected boolean hasScholarship;

    public Student(int studentId, String name, String department, double marks) {
        this.studentId = studentId;
        this.name = name;
        this.department = department;
        this.marks = marks;
        this.hasScholarship = false;
    }

    public Student(int studentId, String name, String department, double marks, boolean hasScholarship) {
        this(studentId, name, department, marks);
        this.hasScholarship = hasScholarship;
    }

    public abstract String calculateGrade() throws DatabaseOperationException;

    public int getStudentId() { return studentId; }
    public String getName() { return name; }
    public String getDepartment() { return department; }

    @Override
    public String toString() {
        return String.format("ID=%-4d | %-15s | Dept=%-12s | Scholarship=%s",
                studentId, name, department, hasScholarship);
    }
}

class UGStudent extends Student {
    public UGStudent(int id, String name, String dept, double marks) {
        super(id, name, dept, marks);
    }
    public UGStudent(int id, String name, String dept, double marks, boolean hasScholarship) {
        super(id, name, dept, marks, hasScholarship);
    }

    @Override
    public String calculateGrade() throws DatabaseOperationException {
        if (marks < 0)
            throw new DatabaseOperationException("Negative or invalid marks error: " + name + " (Marks: " + marks + ")");
        if (marks >= 90) return "O";
        if (marks >= 80) return "A+";
        if (marks >= 70) return "A";
        if (marks >= 60) return "B";
        if (marks >= 50) return "C";
        return "F";
    }

    @Override
    public String toString() {
        return "UGStudent [" + super.toString() + "]";
    }
}

class PGStudent extends Student {
    public PGStudent(int id, String name, String dept, double marks) {
        super(id, name, dept, marks);
    }
    public PGStudent(int id, String name, String dept, double marks, boolean hasScholarship) {
        super(id, name, dept, marks, hasScholarship);
    }

    @Override
    public String calculateGrade() throws DatabaseOperationException {
        if (marks < 0)
            throw new DatabaseOperationException("Negative or invalid marks error: " + name + " (Marks: " + marks + ")");
        if (marks >= 93) return "O";
        if (marks >= 85) return "A+";
        if (marks >= 75) return "A";
        if (marks >= 65) return "B";
        return "F";
    }

    @Override
    public String toString() {
        return "PGStudent [" + super.toString() + "]";
    }
}

class StudentDAO {
    private static final String DB_URL = "jdbc:sqlite:students.db";

    public StudentDAO() {
        try {
```

```
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
        }
    }

    public void initializeDatabase() {
        String dropSQL = "DROP TABLE IF EXISTS students;";
        String createSQL = "CREATE TABLE students (" +
                " student_id INTEGER PRIMARY KEY," +
                " name TEXT," +
                " department TEXT," +
                " type TEXT," +
                " marks REAL," +
                " has_scholarship INTEGER" +
                ");";
        String insertSQL = "INSERT INTO students VALUES (?, ?, ?, ?, ?, ?);";

        try (Connection conn = DriverManager.getConnection(DB_URL);
             Statement stmt = conn.createStatement()) {
            stmt.execute(dropSQL);
            stmt.execute(createSQL);

            try (PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
                Object[][] data = {
                    {101, "Arun Kumar", "CSE", "UG", 88.5, 0},
                    {102, "Bhavna R", "ECE", "UG", 92.0, 1},
                    {103, "Chaitanya S", "AI-DS", "PG", 84.0, 1},
                    {104, "Deepak N", "MECH", "UG", 76.5, 0},
                    {105, "Eve Davis", "IT", "PG", -5.0, 0}
                };
                for (Object[] row : data) {
                    pstmt.setInt(1, (int) row[0]);
                    pstmt.setString(2, (String) row[1]);
                    pstmt.setString(3, (String) row[2]);
                    pstmt.setString(4, (String) row[3]);
                    pstmt.setDouble(5, (double) row[4]);
                    pstmt.setInt(6, (int) row[5]);
                    pstmt.addBatch();
                }
                pstmt.executeBatch();
            }
        } catch (SQLException e) {
        }
    }

    public List<Student> fetchStudentDetails(int studentId) throws DatabaseOperationException {
        System.out.println("[JDBC Query] SELECT * FROM students WHERE student_id = " + studentId);
        List<Student> list = new ArrayList<>();
        String sql = "SELECT * FROM students WHERE student_id = ?;";
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, studentId);
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    list.add(buildStudentFromResultSet(rs));
                }
            }
        } catch (SQLException e) {
        }
        return list;
    }

    public List<Student> fetchStudentDetails(String department) throws DatabaseOperationException {
        System.out.println("[JDBC Query] SELECT * FROM students WHERE department = '" + department + "';");
        List<Student> list = new ArrayList<>();
        String sql = "SELECT * FROM students WHERE department = ?;";
        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, department);
            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    list.add(buildStudentFromResultSet(rs));
                }
            }
        } catch (SQLException e) {
        }
        return list;
    }

    private Student buildStudentFromResultSet(ResultSet rs) throws SQLException, DatabaseOperationException {
        int id = rs.getInt("student_id");
        String name = rs.getString("name");
        String dept = rs.getString("department");
        String type = rs.getString("type");
        double marks = rs.getDouble("marks");
        boolean scholarship = rs.getInt("has_scholarship") == 1;

        if (marks < 0) {
            throw new DatabaseOperationException("Negative or invalid marks error: " + name + " (Marks: " + marks + ")");
        }
        if (type.equals("UG")) {
            return new UGStudent(id, name, dept, marks, scholarship);
        } else {
            return new PGStudent(id, name, dept, marks, scholarship);
        }
    }
}

public class StudentSystem {
    public static void main(String[] args) {
        StudentDAO dao = new StudentDAO();
```

```
        dao.initializeDatabase();

        System.out.println("--- Query 1: Fetch by Student ID (101) ---");
        try {
            List<Student> list = dao.fetchStudentDetails(101);
            for (Student s : list) {
                System.out.println(s);
                System.out.println("Calculated Grade: " + s.calculateGrade());
            }
        } catch (DatabaseOperationException e) {
            System.out.println(" [Validation Error] " + e.getMessage());
        }

        System.out.println("\n--- Query 2: Fetch by Department (CSE) ---");
        try {
            List<Student> list = dao.fetchStudentDetails("CSE");
            for (Student s : list) {
                System.out.println(s);
                System.out.println("Calculated Grade: " + s.calculateGrade());
            }
        } catch (DatabaseOperationException e) {
            System.out.println(" [Validation Error] " + e.getMessage());
        }

        System.out.println("\n--- Query 3: Catch Custom Exception Demo (ID=105) ---");
        try {
            dao.fetchStudentDetails(105);
        } catch (DatabaseOperationException e) {
            System.out.println("Successfully caught exception: " + e.getMessage());
        }
    }
}
```